数据库管理系统原理与实现

杜小勇 陈红 卢卫 主编

深圳技术大学

大数据与互联网学院

3.3 基本SQL查询

- 3.3.1 单表查询
- 3.3.2 连接查询
- 3.3.3 集合查询(Mysql仅支持UNION)
- 3.3.4 空值查询
- 3.3.5 聚集查询

3.3.3 集合查询

若想要查询价格低于50或价格高于1500的商品信息，可以使用以下语句：

```
SELECT * FROM Products WHERE Price <50
UNION SELECT * FROM Products WHERE Price > 1500;
```

集合查询要求两个查询结果有相同数量的列，且对应列的数据类型相同。

3.3.3 集合查询

集合查询要求两个查询结果有相同数量的列，且对应列的数据类型相同。

<non-join query expression> ::=

<non-join query term>

|<query expression> UNION [ALL][<corresponding spec>]<query term>

|<query expression> EXCEPT[ALL][<corresponding spec>]<query term>

UNION表示将两个查询的结果合并。

UNION ALL：将多个查询结果合并起来时，保留重复元组

EXCEPT则表示从第一个查询结果中删去第二个查询的结果(P46:Mysql不支持)。

INTERSECT交操作(Mysql不支持！)

3.3.3 集合查询

[例3.15]查询价格低于50或价格高于1500的商品信息。

```
SELECT * FROM Products WHERE Price <50
UNION SELECT * FROM Products WHERE Price > 1500;
```

查询结果：

PID	PName	Price	Category	SID
P0004	数据库教材	48	书籍	S002
P0001	智能手机	1999	数码产品	S001
P0003	平板电脑	1688	数码产品	S001

Q：用OR如何改写？

```
WHERE Price <50 or Price > 1500
```

集合查询（续）

[例3.66] 查询计算机科学系的学生与年龄不大于19岁的学生 的交集。

```
SELECT *
FROM Student
WHERE Sdept='CS'
INTERSECT
SELECT *
FROM Student
WHERE Sage<=19
```

Q：用AND如何改写？

```
WHERE Sdept= 'CS'
AND  Sage<=19;
```

集合查询（续）

[例,第6版]查询既选修了课程1又选修了课程2的学生。

```
SELECT Sno
 FROM SC
 WHERE Cno=' 1 '
 INTERSECT
 SELECT Sno
 FROM SC
 WHERE Cno='2 ';
```

Q：用AND如何改写？

集合查询（续）

[例]也可以表示为：

```
SELECT Sno
  FROM    SC #Q:sc为何不需要别名?
  WHERE Cno=' 1 ' AND Sno IN
                          (SELECT Sno
                           FROM SC
                           WHERE Cno=' 2 ');
```

A：Where先执行，不相关子查询再执行from

集合查询（续）

[例 3.68] 查询计算机科学系的学生与年龄**不大于**19岁的学生的差集。

```
SELECT *
FROM Student
WHERE Sdept='CS'
EXCEPT
SELECT  *
FROM Student
WHERE Sage <=19;
```

Q：用AND如何改写？

实际上是查询计算机科学系中年龄大于19岁的学生

```
 WHERE Sdept= 'CS'
AND  Sage>19;
```

3.3 基本SQL查询

- 3.3.1 单表查询
- 3.3.2 连接查询
- 3.3.3 集合查询
- 3.3.4 空值查询
- 3.3.5 聚集查询

补充：空值的处理

- 空值就是“不知道”或“不存在”或“无意义”的值。
- 一般有以下几种情况：
  - 该属性应该有一个值，但目前不知道它的具体值
  - 该属性不应该有值
  - 由于某种原因不便于填写

1. 空值的产生

- 空值是一个很特殊的值，含有不确定性。对关系运算带来特殊的问题，需要做特殊的处理。

- 空值的产生

[例]向SC表中插入一个元组，学生号是”201215126”，课程号是”1”，成绩为空。

```
INSERT INTO SC(Sno,Cno,Grade)
VALUES('201215126 ','1',NULL);  /*该学生还没有考试成绩，取空值*/
```

或

```
INSERT INTO SC(Sno,Cno)
VALUES(' 201215126 ','1');          /*没有赋值的属性，其值为空值*/
```

空值的产生（续）

[例] 将Student表中学生号为”201215200”的学生所属的系改为空值（比如转系中）。

```
UPDATE Student
SET Sdept = NULL
WHERE Sno='201215200';
```

2. 空值的判断

判断一个属性是否为空，须使用“**IS NULL**”或“**IS NOT NULL**”。

[例3.16]查询类别为空值的商品。该查询结果为空。

SELECT * FROM Products WHERE Category IS NULL;

查询结果：

PID	PName	Price	Category	SID

3. 空值的约束条件

- 属性定义（或者域定义）中
  - 有NOT NULL约束条件的不能取空值
  - 加了UNIQUE限制的属性不能取空值
  - 码属性不能取空值

4. 空值的算术运算、比较运算和逻辑运算

- 空值与另一个值（包括另一个空值）的算术运算的结果为空值
- 空值与另一个值（包括另一个空值）的比较运算的结果为UNKNOWN。
- 有UNKNOWN后，传统二值（TRUE，FALSE）逻辑就扩展成了三值逻辑

空值的算术运算、比较运算和逻辑运算（续）

[例] 找出选修1号课程的需补考的学生。

```
SELECT Sno
FROM SC
WHERE Grade < 60 AND Cno='1';
```

Q：代码是否正确？

查询结果不包括缓考的学生，因为他们的Grade值为 null。但如果缺考等呢？

空值的算术运算、比较运算和逻辑运算（续）

[例] 选出选修1号课程的不及格的学生以及缺考的学生。

```
SELECT Sno
FROM SC
WHERE Grade < 60 AND Cno='1'
UNION
SELECT Sno
FROM SC
WHERE Grade IS NULL AND Cno='1'
```

或者

```
SELECT Sno
FROM SC
WHERE Cno='1' AND (Grade<60 OR Grade IS NULL);
```

3.3 基本SQL查询

- 3.3.1 单表查询
- 3.3.2 连接查询
- 3.3.3 集合查询
- 3.3.4 空值查询
- 3.3.5 聚集查询

- 聚集函数：
  - 统计元组个数： COUNT(*)
  - 统计某列中值的个数： COUNT([DISTINCT|ALL] <列名>）注意是否去重

count(*)符合SQL标准，会统计值为 NULL 的行(count(1)也可以)，而 count(列名)不会统计此列为 NULL 值的行。

  - 计算一列值的总和（此列必须为数值型）

    SUM([DISTINCT|ALL] <列名>)　一般选择不去重
  - 计算一列值的平均值（此列必须为数值型）

    AVG([DISTINCT|ALL] <列名>) 一般不去重
  - 求一列中的最大值和最小值(最好用于数值型)

    MAX([DISTINCT|ALL] <列名>) 一般不去重

    MIN([DISTINCT|ALL] <列名>) 一般不去重

聚集函数（续）

[例,第6版] 查询学生总人数。

```
SELECT COUNT(*)
FROM  Student;
```

[例,第6版] 查询选修了课程的学生人数。

```
SELECT COUNT(DISTINCT Sno)
FROM SC;
```

[例,第6版] 计算1号课程的学生平均成绩。

```
SELECT round(AVG(Grade),2)
 FROM    SC
 WHERE Cno= ' 1 ';
```

AVG的精度一般要代码控制

round(number,decimals)

number:要四舍五入的数字

decimals:要保留的小数位数

聚集函数 （续）

[例,第6版] 查询选修1号课程的学生最高分数。

```
SELECT MAX(Grade)
FROM SC
WHERE Cno='1';
```

[例,第6版] 查询学生201215012选修课程的总学分数。

```
SELECT SUM(Ccredit)
FROM  SC,Course
WHERE Sno='201215012' AND
SC.Cno=Course.Cno;
```

Limit

SELECT * FROM table LIMIT [offset,] rows | rows OFFSET of fset

第一个参数指定第一个返回记录行的偏移量，第二个参数指定返回记录行的最大数目。初始记录行的偏移量是0(而不是 1)

SELECT * FROM SC LIMIT 1; //检索前 1 个记录行

SELECT * FROM SC LIMIT 5,10; # 检索记录行 6-15

SELECT * FROM SC LIMIT 95,-1; # 检索记录行 96-last

Limit

[例,第6版]查询选修1号课程的学生最高分数。

```
SELECT Sno, Grade
FROM    SC
WHERE  Cno= ' 1 '
ORDER BY Grade DESC
 LIMIT 1;
```

两种方法是否等效？

```
SELECT Sno, MAX(Grade)
  FROM SC
  WHERE Cno='1';
```

- GROUP BY子句分组：

  细化聚集函数的作用对象

  - 如果未对查询结果分组，聚集函数将作用于整个查询结果
  - 对查询结果分组后，聚集函数将分别作用于每个组
  - 按指定的一列或多列值分组，值相等的为一组(多列比如jno,sno的一个取值为一组， pno可以不同,而order by不分组)。

GROUP BY子句（续）

[例3.46] 求各个课程号及相应的选课人数。

```
SELECT Cno, COUNT(Sno)
FROM   SC
GROUP BY Cno;
```

查询结果示例：

Cno	COUNT(Sno)
1	22
2	34
3	44
4	33
5	48

GROUP BY子句（续）

[例,第6版] 查询选修了3门以上课程的学生学号。

```
SELECT Sno
FROM   SC
GROUP BY Sno
HAVING  COUNT(*) >3; #针对sno分组
```



Q：如何修改本例为查询选修了所有课程的学生学号?

```
HAVING  COUNT(*) =select COUNT(*) from COURSE
```

GROUP BY子句（续）

[例] 查询选修了3门及以上课程的学生学号。

SC表

Sno	Cno	Grade
200215121	1	92
200215121	2	85
200215121	3	88
200215122	2	90
200215122	3	80

HAVING短语与WHERE子句的区别：

HAVING短语作用于组，从中选择满足条件的组；

WHERE子句作用于基本表或视图，从中选择满足条件的元组。

HAVING

GROUP BY

Sno	COUNT(*)
200215121	3
200215122	2

结果

Sno
200215121

中国人民大学 数据库系统概论

GROUP BY子句（续）

- HAVING短语与WHERE子句的区别：
  - 作用对象不同
  - WHERE子句作用于基表或视图，对分组前的数据过滤，不可含聚集函数
  - HAVING短语作用于group by的组(选定的列)，对分组后的数据过滤，可包含聚集函数。

GROUP BY子句（续）

[课堂练习] 查询每门课(Cno)的最高分。

```
SELECT Cno, MAX(Grade)
FROM SC
GROUP BY Cno
```

GROUP BY子句（续）

```
SELECT Cno, Sno /*Q：能不能列出每门课所有选课的学生？*/
  FROM SC
  GROUP BY Cno;
```

按cno分组，因此每个cno只能一行。
改为：SELECT Cno, GROUP_CONCAT(Sno)。。。
Sno逗号分隔，合并为一行字符串，但还需特殊处理，
不推荐。

GROUP_CONCAT函数用于将查询结果按指定顺序连接成一个字符串

```
select cno, sno
  from SC
  order by cno,sno; /*先按cno升序，再按sno升序*/
```

GROUP BY子句（续）

[例,第6版]查询平均成绩大于等于90分的学生学号和平均成绩

下面的语句是不对的：

```
SELECT Sno, AVG(Grade)
FROM  SC
WHERE AVG(Grade)>=90
GROUP BY Sno;
```

因为WHERE子句中是不能用聚集函数作为条件表达式，可以用子查询来实现（在子查询中进行聚合函数计算，然后在主查询中进行where判断）

正确的查询语句应该是：

```
SELECT  Sno, AVG(Grade)
FROM  SC
GROUP BY Sno
HAVING AVG(Grade)>=90;  #按sno分组的一到多行求AVG
```

GROUP BY子句（续）

- group by后不可以接别名(编译报错)
- select 后出现的非聚集函数列，包含(即子集)在group by后出现的列 (如果select后的非聚集函数列，但不与group by后的列对应，则由于group by的分组，导致数据不符合预期！但编译不报错)。
- group by后不能使用where(编辑器会报错)，因为where是在分组之前起作用的，分组后的数据进行过滤只能使用having
- group by之前可使用where过滤数据

MySQL GROUP BY子句

```
SELECT column_name, function(column_name)
 FROM table_name
WHERE column_name operator value
GROUP BY column_name;
```

3.3.5 聚集查询

在查询定义的<value expression>中，可以使用聚集函数 。如查询不同种类的商品的数量，可使用以下语句

SELECT Category, COUNT(*) AS 数量 FROM Products GROUP BY Category;

<set function specification> ::=

COUNT <left paren> <asterisk> <right paren> | <general set function>

3.3.5 聚集查询

<general set function> ::=

<set function type> <left paren> [<set quantifier>] <value expression> <right paren>

SQL标准包含五个聚集函数，分别是AVG、MAX、MIN、SUM和COUNT。

<set function type> ::= AVG | MAX | MIN | SUM | COUNT

指定DISTINCT可以去掉重复值。默认是ALL，即所有值参与运算。

<set quantifier> ::= DISTINCT | ALL

量词

3.3.5 聚集查询

[例3.17]查询最高的商品价格。

```
SELECT MAX(Price) AS 最高价格 FROM Products;
```

查询结果：

最高价格
1999

3.3.5 聚集查询

[例3.18]查询不同种类的商品的数量 。

```
SELECT Category, COUNT(*) AS 数量 FROM Products GROUP BY
Category;
```

查询结果：

Category	数量
数码产品	3
书籍	2

插入子查询结果（续）

[例3.20]新增一个关系TotalPrice(不讲其合理性)，记录各订单的总价，然后根据Products和OrderItems关系查询所有订单的总价。

```
CREATE TABLE TotalPrice(
        OID VARCHAR(32) PRIMARY KEY,
        TotalPrice DECIMAL
);
INSERT INTO TotalPrice (OID, TotalPrice)
SELECT OID, SUM(Quantity * Price * Discount)
        FROM Products JOIN OrderItems
        ON Products.PID = OrderItems.PID GROUP BY OID;
```

创建了一个关系TotalPrice，包含OID和TotalPrice两个属性。

执行一个插入语句，该语句将里面的子查询结果插入到TotalPrice中。

该子查询连接Products和OrderItems，并按OID进行分组，最后计算每组的数量乘以价格再乘以折扣之和，得到订单总价。

SQL语句的执行顺序

SQL语句的执行顺序一般为：

from子句

where 子句

group by 子句

having 子句

order by 子句

select 子句

聚集函数也叫列函数，它们都是基于整列数据进行计算的，where子句针对行操作，还处于“确定”结果集的过程中，因而不能使用聚集函数

SQL语句的执行顺序

MySQL语句的执行顺序一般为：

1. FROM 对FROM的多表计算笛卡尔积。产生虚表VT1
2. WHERE 对虚拟表VT2进行WHERE条件过滤，符合的记录插入到虚拟表VT3中。或者：JOIN ON 对虚表VT1进行筛选，符合的行记录在虚表VT2。性能优于from+where
3. GROUP BY 对VT3中的记录进行分组操作，产生VT4
4. HAVING 对虚拟表VT4的按列分组应用having过滤，符合的记录插入到虚拟表VT5。
5. ORDER BY 将虚拟表VT5中的记录进行排序操，产生虚拟表VT6.
6. SELECT 执行select操作，选择指定的列，插入到虚拟表VT7中。
7. DISTINCT 对VT7中的记录进行去重。产生虚拟表VT8.
8. limit 取出指定行的记录，产生虚拟表VT9, 并将结果返回

OJ实验涉及的表结构：第6版教材P65/pdf p90

6. 设有一个 SPJ 数据库，包括 4 个关系模式 S、P、J 和 SPJ。

S(SNO,SNAME,STATUS,CITY)；

P(PNO,PNAME,COLOR,WEIGHT)；

J(JNO,JNAME,CITY)；

SPJ(SNO,PNO,JNO,QTY)。

供应商表 S 由供应商代码(SNO)、供应商姓名(SNAME)、供应商状态(STATUS)、供应商所在(CITY)组成。

零件表 P 由零件代码(PNO)、零件名(PNAME)、颜色(COLOR)、重量(WEIGHT)组成。

工程项目表 J 由工程项目代码(JNO)、工程项目名(JNAME)、工程项目所在城市(CITY)组成。

供应情况表 SPJ 由供应商代码(SNO)、零件代码(PNO)、工程项目代码(JNO)、供应数量(QTY)组成，表示某供应商供应某种零件给某工程项目的数量为 QTY。

Q：以下代码有几处问题？

```
SELECT * FROM spj where SNO=s2 GROUP by SNO as a LIMIT 0, 25
```

只能显示分组中的一行

此处不能用别名

MySQL 返回： sno定义是字符串

```
#1064 - You have an error in your SQL syntax; check the manual that corresponds to your
MySQL server version for the right syntax to use near 'as a LIMIT 0, 25' at line 1
```

```
SELECT * FROM spj where SNO='s2' GROUP by SNO
```

☐ 性能分析 [编辑内嵌] [

☐ 显示全部 | 行数: 25 过滤

+ 选项

SNO	PNO	JNO	QTY
S2	P3	J1	400

GROUP BY子句（续）

```
SELECT cno, sno, AVG(grade) FROM sc WHERE cno='001' GROUP by cno
```

select显示其他列并不报错，但会导致误导
(非聚集函数)

显示全部 | 行数: 25 过滤行: 在表中搜索

+ 选项

	cno	sno	AVG(grade)
编辑 复制 删除	001	1	86.0000

GROUP BY子句（续）

```
1  SELECT sno,SUM(qty) FROM spj where having SNO='s2'
   GROUP by SNO
```

静态分析： Having仅用于group by后

分析时发现1个错误。

1. 错误的语句顺序。 (near "having" at position 24)

SQL 查询：

SELECT * FROM spj where having SNO='s2' GROUP by SNO

MySQL 返回：

#1064 - You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'having SNO='s2' GROUP by SNO' at line 1

GROUP BY子句（续）

```
SELECT sno,SUM(qty) FROM spj where SUM(qty)>1000  GROUP by SNO
```

MySQL 返回：

where筛选执行之后才能用聚集函数，聚集函数用于select或having后

```
#1111 - Invalid use of group function
```